

```
Консоль отладки Microsoft Vi x + v

=== BIKE SHOP (LAB 2) ===
1. Сгенерувати 1000 велосипедів (Паралельно)
2. Показати статистику (LINQ)
3. Вихід
Ваш вибір: 1
Генерація даних...
Готово! Створено за 1290 мс.
Дані збережено у data.json

=== BIKE SHOP (LAB 2) ===
1. Сгенерувати 1000 велосипедів (Паралельно)
2. Показати статистику (LINQ)
3. Вихід
Ваш вибір: 2
--- Статистика цін ---
Мінімальна: 500,66
Максимальна: 4997,04
Середня: 2752,83

=== BIKE SHOP (LAB 2) ===
1. Сгенерувати 1000 велосипедів (Паралельно)
2. Показати статистику (LINQ)
3. Вихід
Ваш вибір: 3

C:\Users\User\Documents\GitHub\NUPP_NET_2026_301_TK_Skliarov_LAB\BikeService.Console\bin\Debug\net8.0\BikyShop.Console.exe (процесс 1116) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно.
```

```
C: > Users > User > Documents > GitHub > NUPP_NET_2026_301_TK_Skliarov_LAB > BikeService.Console > bin > Debug > net8.0 > data.json > ...

1  {
2  {
3      "Type": "Mountain",
4      "Id": "07679abb-2d38-49d5-a83c-08785b184ddb",
5      "Brand": "Cannondale",
6      "Model": "B-847",
7      "Price": 4697.55034165197
8  },
9  {
10     "Type": "City",
11     "Id": "478108e9-d1df-4e71-b903-aa283f9f3f5b",
12     "Brand": "Specialized",
13     "Model": "B-293",
14     "Price": 2584.45022466906
15   },
16   {
17     "Type": "City",
18     "Id": "49f0a58a-c268-41f1-b53e-4957aba4699b",
19     "Brand": "Cannondale",
20     "Model": "B-817",
21     "Price": 1783.51268245581
22   },
23   {
24     "Type": "City",
25     "Id": "e1e00262-33e2-4cd3-b1f7-9d25ebb9add5",
26     "Brand": "Cannondale",
27     "Model": "B-230",
28     "Price": 3722.8144809269
29   },
30   {
31     "Type": "Touring",
32     "Id": "7748ecea-0fcc-4725-87bb-4fc91ca201d8",
33     "Brand": "Giant",
```

```
3393     "Brand": "Cannondale",
3394     "Model": "B-139",
3395     "Price": 1139.75791838191
3396   },
3397   {
3398     "Type": "Mountain",
3399     "Id": "e93caf8d-8809-475a-87e9-036dd74fec67",
3400     "Brand": "Giant",
3401     "Model": "B-497",
3402     "Price": 1244.17772344482
3403   },
3404   {
3405     "Type": "Road",
3406     "Id": "8d76573e-b88f-461c-b009-7c1f9de0e093",
3407     "Brand": "Giant",
3408     "Model": "B-627",
3409     "Price": 2792.67592661446
3410   },
3411   {
3412     "Type": "City",
3413     "Id": "7495da03-1698-43f1-a770-689526788eas",
3414     "Brand": "Cannondale",
3415     "Model": "B-729",
3416     "Price": 1747.15578269955
3417   },
3418   {
3419     "Type": "Touring",
3420     "Id": "148a1f35-ef4e-40e5-834d-5926ca2b5353",
3421     "Brand": "Cannondale",
3422     "Model": "B-524",
3423     "Price": 2671.38536962506
```

КОНТРОЛЬНІ ЗАПИТАННЯ

1. Що таке інтерфейс ICrudServiceAsync? Яке його призначення? Чому доцільно використовувати generic тип T для CRUD-сервісу?

Це контракт, який визначає універсальний набір асинхронних методів для керування даними, а використання generic-типу T дозволяє створити єдину логіку для будь-якої сутності, що усуває дублювання коду, забезпечує типобезпеку та значно спрощує підтримку архітектури системи.

2. Що таке процес та потік, яка між ними різниця? Що таке багатопотокове виконання?

Процес — це окремий екземпляр програми з власною ізольованою пам'яттю, тоді як потік — це найменша одиниця виконання всередині процесу, що ділить ресурси з іншими потоками того ж процесу; багатопотокове виконання дозволяє системі одночасно виконувати декілька операцій, що суттєво підвищує продуктивність при виконанні складних задач.

3. Що таке асинхронність, яка різниця між асинхронністю та багатопотоковістю?

Асинхронність дозволяє програмі не блокувати виконання при очікуванні відповіді від зовнішніх ресурсів, наприклад БД чи мережі, на відміну від багатопотоковості, яка фокусується на розпаралелюванні обчислювальних задач між ядрами процесора, щоб вони виконувалися фізично одночасно.

4. Для чого використовуються ключові слова `async/await`?

Вони служать для написання асинхронного коду в зручному послідовному вигляді, де `async` позначає метод як асинхронний, а `await` призупиняє виконання методу до завершення операції, звільняючи потік для інших задач до моменту отримання результату.

5. Чим відрізняється `Task` від `ValueTask`?

`Task` є повноцінним об'єктом у пам'яті (reference type), що має певні витрати ресурсів, тоді як `ValueTask` — це структура (value type), яку доцільно використовувати для оптимізації продуктивності, якщо операція часто завершується синхронно і миттєво.

6. Що таке `thread-safe` колекція? Які приклади таких колекцій у .NET ви знаєте?

Це спеціальні структури даних, спроектовані для коректної роботи з багатьох потоків одночасно без ручного блокування, до них належать `ConcurrentDictionary`, `ConcurrentQueue`, `ConcurrentStack` та інші, які автоматично вирішують конфлікти при паралельному записі чи читанні.

7. Для чого використовуються примітиви синхронізації, такі як `lock`, `Semaphore`, `AutoResetEvent`?

Вони потрібні для координації доступу до спільних ресурсів, де `lock` забезпечує виключне право доступу для одного потоку, `Semaphore` дозволяє обмежити кількість потоків, що одночасно працюють із ресурсом, а

AutoResetEvent використовується для того, щоб один потік міг сигналізувати іншому про настання певної події.

8. Як забезпечити безпеку при одночасному зверненні кількох потоків до спільного ресурсу?

Безпеку можна забезпечити через використання примітивів синхронізації для створення критичних секцій, де виконується робота з ресурсом, або ж через застосування потокобезпечних колекцій, які самі беруть на себе управління доступом у багатопотоковому середовищі.

9. Як за допомогою LINQ можна отримати мінімальне, максимальне та середнє значення певної властивості?

Для цього використовуються методи розширення Min, Max та Average, які застосовуються до колекції з лямбда-виразом, що вказує на конкретну властивість об'єкта, наприклад `list.Average(x => x.Price)`.

10. У чому різниця між методами Select, Where, Aggregate, OrderBy?
Select трансформує кожен елемент послідовності, Where фільтрує елементи за умовою, Aggregate зводить усі елементи до одного результату шляхом поступового накопичення, а OrderBy впорядковує дані згідно з вказаним ключем.

11. Які переваги використання LINQ у порівнянні з класичними циклами?

LINQ забезпечує декларативну форму опису маніпуляцій із даними, що робить код значно лаконічнішим, читабельнішим та менш схильним до помилок, дозволяючи абстрагуватися від внутрішньої реалізації перебору даних.

12. Що станеться, якщо два потоки одночасно спробують зберегти колекцію у файл?

Без використання синхронізації виникне виключення `IOException` через те, що файл буде заблокований, або дані будуть пошкоджені в результаті одночасного запису, тому для таких операцій обов'язково застосовувати lock або механізм черг.

13. Як працює Parallel.For і у яких випадках його краще використовувати?

Parallel.For автоматично розділяє ітерації циклу між доступними процесорними ядрами, тому його краще використовувати для ресурсомістких обчислювальних задач, де кожна ітерація є незалежною від інших і не потребує складної синхронізації.

14. Що таке пагінація і як вона реалізована у вашій системі?

Пагінація — це техніка вибірки даних порціями, що дозволяє уникнути надмірного навантаження на пам'ять та інтерфейс, реалізується вона через

методи Skip (який пропускає кількість записів, що відповідають попереднім сторінкам) та Take (який вибирає лише потрібну кількість записів для поточної сторінки).